Review notes · for you

# Agent engineering: planning, multi-agent, evals and guardrails: unit check

AI agents · University · Unit check

| Time | Materials |
| --- | --- |
| 20 min | Paper and a pen. No device needed. |

## Learning goals

- Plans and memory
- Several agents, one job
- Measuring an agent
- Fine-tuning and guardrails

## Activity: Agent design review

1. Warm-up (3 min): Write one idea from each of the four lessons, in your own words.
2. Design (8 min): Sketch an agent for a real task at your university, such as answering course questions or booking rooms: its steps, what it keeps in its window, its roles and its tools.
3. Review (7 min): Swap with a partner. Check their design against the unit: where would errors carry forward, what could fall out of memory, how would they test it, and which actions are guarded in code?
4. Share (2 min): Each pair shares the weakest point they found and how they would fix it.

**Use made-up names and data in every design. The point is the review, not a perfect agent.**

## Discussion

- Which idea from this unit changes how you would build an agent?
- What would convince you that an agent is ready for real users?
- Which guard would you add first to an AI tool you already use?

## Check yourself

- My most useful idea from this unit + the project I will use it in.

**Try it this week**
Pick the habit from this unit you need most (checking every step, pinning key facts, a fixed test set or exact approval in code) and use it on a real project this week.

Quiz sheet

# Agent engineering: planning, multi-agent, evals and guardrails: unit check

AI agents · University · Unit check

Name _____ Class _____ Date _____

**1** **Each step of a four-step agent is right 90% of the time. About how often is the whole task right?**

◯ About 66% ◯ About 90% ◯ About 36%

**2** **A model's context window is its only memory within a conversation: what falls out of it is forgotten.**

◯ True ◯ False

**3** **Which help an agent keep what matters as a conversation grows? Choose all that apply.**

☐ Keeping everything until the window overflows
☐ Short notes in place of finished exchanges
☐ Retrieving only the part of a document a step needs
☐ Pinning the key facts, such as dates and places
☐ Dropping the oldest turns, with no notes

**4** **Match each role in a team agent to its job.**

| | | |
|---|---|---|
| Planner | • • • • | Runs the tool and returns the result |
| Checker | • • • • | Approves actions and handles what's stopped |
| Runner | • • • • | Proposes the next tool call |
| Person | • • • • | Tests the call against strict rules |

**5** **A test set scored by group. What does this program print?**

```
hits = {"en": 15, "ar": 14}
total = {"en": 16, "ar": 16}
print(hits["ar"] / total["ar"])
```

◯ 0.875 ◯ 0.9375 ◯ 14

**6** **Put the steps of an honest evaluation in order.**

☐ Run the agent and score every case exactly
☐ Re-run the same set after every change
☐ Decide the right answers before running
☐ Collect requests, including held-out wordings
☐ Break the score down and read the failures

**7** **An agent that scores 16 out of 16 on wordings it was trained on will do about as well on new wordings.**

◯ True ◯ False

**8** **Complete the approval gate: it runs a call only if it is exactly the approved one, and logs every decision.**

```
def gate(call, approved, log):
    ok = call _____ approved
    log._____((call["tool"], ok))
    if ok:
        return "run"
    return "ask again"
```

**Word bank:** append pop == !=

**9** **A fine-tune with no old conversations mixed in gets 7/8 on the new bus tool but 1/8 on old skills. What is happening?**

◯ Catastrophic forgetting: the new learning overwrote old skills

◯ The bus tool is simply harder than the others ◯ The test set is too small to show anything

**10** **Which of these still protect you if the model's learned safety fails? Choose all that apply.**

☐ The refusals it learned in training ☐ Comparing each call with the approved call

☐ Only the tools the task needs, enforced in code ☐ A log of every action

☐ Telling the model in its instructions to be careful

Check your answers

# Agent engineering: planning, multi-agent, evals and guardrails: unit check

AI agents · University · Unit check

1 **About 66%**

The chances multiply: 0.9 × 0.9 × 0.9 × 0.9 ≈ 0.66.

2 **True**

The model sees only the tokens in its window (384 for Darsi Tiny).

3 **Short notes in place of finished exchanges; Pinning the key facts, such as dates and places; Retrieving only the part of a document a step needs**

Notes, pinned facts and retrieval keep the essentials in fewer tokens.

4 **Planner — Proposes the next tool call**
**Checker — Tests the call against strict rules**
**Runner — Runs the tool and returns the result**
**Person — Approves actions and handles what's stopped**

Splitting the work lets one part check another before anything happens: the model only proposes, code checks and runs, and a person keeps the final say on actions.

5 **0.875**

The Arabic group has 14 hits out of 16 cases, and 14 / 16 = 0.875.

6 **Top to bottom: 3, 5, 2, 1, 4**

Answers are fixed before the run so they can't be bent to fit, scoring is exact, failures are read by group, and the same set is re-run to catch regressions.

7 **False**

Trained wordings measure memory: Darsi Tiny drops from 16/16 to 13/16 on new wordings, and to 78–80% across thousands of them.

8

```
def gate(call, approved, log):
    ok = call == approved
    log.append((call["tool"], ok))
    if ok:
        return "run"
    return "ask again"
```

== compares the whole call, tool and every detail, with the approved one; append adds the decision to the log, whether it ran or not.

9 **Catastrophic forgetting: the new learning overwrote old skills**

Training only on the new examples changed numbers the old skills relied on.

10 **Only the tools the task needs, enforced in code; Comparing each call with the approved call; A log of every action**

Guards in code hold whatever the model writes.